

Optimizing A* Search with BFS and MST Heuristics for Multi-Goal Path Planning

This project implements an optimized A* search with a custom heuristic that combines BFS distance precomputation and MST approximation, significantly reducing search cost in large environments.

This project focuses on improving search efficiency in complex path planning problems with multiple required targets.

1. Problem Overview

This project focuses on solving a multi-goal path planning problem (CheeseHunter environment) using search algorithms.

The environment is a grid. The agent needs to activate all levers and then reach the goal.

Each state includes:

- The agent's position
- The activation status of all traps

The goal is to find the lowest-cost sequence of actions to complete the task.

2. Approach

I implemented and compared two algorithms:

- **Uniform Cost Search (UCS)** — expands nodes based on path cost
- **A*** — expands nodes based on $f(n)=g(n)+h(n)$

The main difference is the heuristic function $h(n)$, which estimates the remaining cost.

The goal is to evaluate how much a well-designed heuristic can improve search efficiency compared to uninformed search.

3. Heuristic Design

The key part of this project is the heuristic for A*.

Instead of using a simple distance like Manhattan, I designed a heuristic that takes obstacles and multiple targets (levers) into account.

BFS distances

I first precomputed shortest distances using BFS:

- from every position to the goal
- from every position to each lever

This makes the heuristic aware of walls and obstacles.

Visiting all levers

Since all levers must be activated, I estimated this cost using a minimum spanning tree (MST) over the remaining levers.

This gives a reasonable lower bound of the total path needed.

Connecting to the goal

After activating the levers, the agent still needs to reach the goal. So I added the minimum distance from any remaining lever to the goal.

Final heuristic

The heuristic combines:

- distance to reach the lever set
- cost to visit all levers (MST)

- cost to reach the goal

Then I scale it by the minimum action cost so it stays admissible. Overall, the heuristic provides a tighter lower bound than simple distance metrics, while remaining admissible.

4. Performance

I tested both UCS and A* on multiple maps.

For small maps, A* doesn't help much because computing the heuristic also costs time.

But for larger maps, the difference is clear.

For the largest test case:

- UCS explored over **4.2 million nodes** and took about **120 seconds**
- A* explored about **67k nodes** and took around **4.5 seconds**

So A* reduced the search space a lot and ran much faster. This shows that the heuristic effectively reduces unnecessary exploration.

5. Summary

This project shows how much difference a good heuristic can make.

- A simple search like UCS works but scales badly
- A better heuristic can reduce both time and memory significantly

At the same time, there is a trade-off: a stronger heuristic usually takes more effort to compute.

This highlights the importance of problem-specific heuristic design in practical search problems.